

A4-R.R

ujjwa

2025-06-30

```
# Main Imports -----
# Set the base directory for the project
base_dir <- getwd()
setwd(base_dir)

# Define a helper function to install packages if not already installed
install <- function(pkg) {
  if (!require(pkg, character.only = TRUE)) {
    install.packages(pkg, dependencies = TRUE, quiet = TRUE)
  }
}

# Define a helper function to load packages
load_pkg <- function(pkg) {
  message(paste("Loading package:", pkg))
  library(pkg, character.only = TRUE, quietly = TRUE)
}

# Required packages for this analysis
pkgs <- c(
  "dplyr",
  "tidyr",
  "ggplot2",
  "caret",
  "e1071",
  "ROCR",
  "rpart",
  "rpart.plot",
  "MASS"
)

# Apply install and load functions
lapply(pkgs, install)
```

```
## Loading required package: dplyr
##
## Attaching package: 'dplyr'
## The following objects are masked from 'package:stats':
##
##   filter, lag
## The following objects are masked from 'package:base':
```

```

##
## intersect, setdiff, setequal, union
## Loading required package: tidyr
## Loading required package: ggplot2
## Loading required package: caret
## Loading required package: lattice
## Loading required package: e1071
## Loading required package: ROCR
## Loading required package: rpart
## Loading required package: rpart.plot
## Loading required package: MASS
##
## Attaching package: 'MASS'
## The following object is masked from 'package:dplyr':
##
## select
## [[1]]
## NULL
##
## [[2]]
## NULL
##
## [[3]]
## NULL
##
## [[4]]
## NULL
##
## [[5]]
## NULL
##
## [[6]]
## NULL
##
## [[7]]
## NULL
##
## [[8]]
## NULL
##
## [[9]]
## NULL
lapply(pkgs, load_pkg)

## Loading package: dplyr
## Loading package: tidyr
## Loading package: ggplot2

```

```

## Loading package: caret
## Loading package: e1071
## Loading package: ROCR
## Loading package: rpart
## Loading package: rpart.plot
## Loading package: MASS

## [[1]]
## [1] "MASS"      "rpart.plot" "rpart"      "ROCR"      "e1071"
## [6] "caret"     "lattice"    "ggplot2"    "tidyr"     "dplyr"
## [11] "stats"     "graphics"   "grDevices"  "utils"     "datasets"
## [16] "methods"   "base"
##
## [[2]]
## [1] "MASS"      "rpart.plot" "rpart"      "ROCR"      "e1071"
## [6] "caret"     "lattice"    "ggplot2"    "tidyr"     "dplyr"
## [11] "stats"     "graphics"   "grDevices"  "utils"     "datasets"
## [16] "methods"   "base"
##
## [[3]]
## [1] "MASS"      "rpart.plot" "rpart"      "ROCR"      "e1071"
## [6] "caret"     "lattice"    "ggplot2"    "tidyr"     "dplyr"
## [11] "stats"     "graphics"   "grDevices"  "utils"     "datasets"
## [16] "methods"   "base"
##
## [[4]]
## [1] "MASS"      "rpart.plot" "rpart"      "ROCR"      "e1071"
## [6] "caret"     "lattice"    "ggplot2"    "tidyr"     "dplyr"
## [11] "stats"     "graphics"   "grDevices"  "utils"     "datasets"
## [16] "methods"   "base"
##
## [[5]]
## [1] "MASS"      "rpart.plot" "rpart"      "ROCR"      "e1071"
## [6] "caret"     "lattice"    "ggplot2"    "tidyr"     "dplyr"
## [11] "stats"     "graphics"   "grDevices"  "utils"     "datasets"
## [16] "methods"   "base"
##
## [[6]]
## [1] "MASS"      "rpart.plot" "rpart"      "ROCR"      "e1071"
## [6] "caret"     "lattice"    "ggplot2"    "tidyr"     "dplyr"
## [11] "stats"     "graphics"   "grDevices"  "utils"     "datasets"
## [16] "methods"   "base"
##
## [[7]]
## [1] "MASS"      "rpart.plot" "rpart"      "ROCR"      "e1071"
## [6] "caret"     "lattice"    "ggplot2"    "tidyr"     "dplyr"
## [11] "stats"     "graphics"   "grDevices"  "utils"     "datasets"
## [16] "methods"   "base"
##
## [[8]]
## [1] "MASS"      "rpart.plot" "rpart"      "ROCR"      "e1071"
## [6] "caret"     "lattice"    "ggplot2"    "tidyr"     "dplyr"

```

```

## [11] "stats"      "graphics"   "grDevices"  "utils"      "datasets"
## [16] "methods"    "base"
##
## [[9]]
## [1] "MASS"      "rpart.plot" "rpart"      "ROCR"       "e1071"
## [6] "caret"     "lattice"    "ggplot2"    "tidyr"      "dplyr"
## [11] "stats"     "graphics"   "grDevices"  "utils"      "datasets"
## [16] "methods"    "base"

# Global File Config -----
datasets_dir <- "datasets"
datasets_path <- file.path(base_dir, datasets_dir)
file_name <- "Credit Card Defaulter Prediction.csv"
file_path <- file.path(datasets_path, file_name)

# Part A: Data Preprocessing and Model Training (Logistic Regression & Decision Tree) -----

# Data Loading
df_a <- read.csv(file_path)

# Initial Data Inspection and Cleaning -----
cat("Initial Data Head:\n")

## Initial Data Head:
print(head(df_a))

##   ID LIMIT_BAL SEX      EDUCATION MARRIAGE AGE PAY_0 PAY_2 PAY_3 PAY_4 PAY_5
## 1  1    20000  F      University   Married  24    2    2   -1   -1   -2
## 2  2   120000  F      University   Single  26   -1    2    0    0    0
## 3  3    90000  F      University   Single  34    0    0    0    0    0
## 4  4    50000  F      University   Married  37    0    0    0    0    0
## 5  5    50000  M      University   Married  57   -1    0   -1    0    0
## 6  6    50000  M Graduate school   Single  37    0    0    0    0    0
##   PAY_6 BILL_AMT1 BILL_AMT2 BILL_AMT3 BILL_AMT4 BILL_AMT5 BILL_AMT6 PAY_AMT1
## 1    -2     3913     3102      689         0         0         0         0
## 2     2     2682     1725     2682     3272     3455     3261         0
## 3     0    29239    14027    13559    14331    14948    15549    1518
## 4     0    46990    48233    49291    28314    28959    29547    2000
## 5     0     8617     5670    35835    20940    19146    19131    2000
## 6     0    64400    57069    57608    19394    19619    20024    2500
##   PAY_AMT2 PAY_AMT3 PAY_AMT4 PAY_AMT5 PAY_AMT6 default
## 1     689         0         0         0         0      Y
## 2    1000     1000     1000         0     2000      Y
## 3    1500     1000     1000     1000     5000      N
## 4    2019     1200     1100     1069     1000      N
## 5   36681    10000     9000     689     679      N
## 6    1815      657     1000     1000     800      N

cat("\nData Structure:\n")

##
## Data Structure:
str(df_a)

## 'data.frame':   30000 obs. of  25 variables:

```

```
## $ ID      : int  1 2 3 4 5 6 7 8 9 10 ...
## $ LIMIT_BAL: int 20000 120000 90000 50000 50000 50000 500000 100000 140000 20000 ...
## $ SEX      : chr  "F" "F" "F" "F" ...
## $ EDUCATION: chr  "University" "University" "University" "University" ...
## $ MARRIAGE : chr  "Married" "Single" "Single" "Married" ...
## $ AGE      : int  24 26 34 37 57 37 29 23 28 35 ...
## $ PAY_0    : int  2 -1 0 0 -1 0 0 0 0 -2 ...
## $ PAY_2    : int  2 2 0 0 0 0 0 -1 0 -2 ...
## $ PAY_3    : int -1 0 0 0 -1 0 0 -1 2 -2 ...
## $ PAY_4    : int -1 0 0 0 0 0 0 0 0 -2 ...
## $ PAY_5    : int -2 0 0 0 0 0 0 0 0 -1 ...
## $ PAY_6    : int -2 2 0 0 0 0 0 -1 0 -1 ...
## $ BILL_AMT1: int 3913 2682 29239 46990 8617 64400 367965 11876 11285 0 ...
## $ BILL_AMT2: int 3102 1725 14027 48233 5670 57069 412023 380 14096 0 ...
## $ BILL_AMT3: int 689 2682 13559 49291 35835 57608 445007 601 12108 0 ...
## $ BILL_AMT4: int 0 3272 14331 28314 20940 19394 542653 221 12211 0 ...
## $ BILL_AMT5: int 0 3455 14948 28959 19146 19619 483003 -159 11793 13007 ...
## $ BILL_AMT6: int 0 3261 15549 29547 19131 20024 473944 567 3719 13912 ...
## $ PAY_AMT1 : int 0 0 1518 2000 2000 2500 55000 380 3329 0 ...
## $ PAY_AMT2 : int 689 1000 1500 2019 36681 1815 40000 601 0 0 ...
## $ PAY_AMT3 : int 0 1000 1000 1200 10000 657 38000 0 432 0 ...
## $ PAY_AMT4 : int 0 1000 1000 1100 9000 1000 20239 581 1000 13007 ...
## $ PAY_AMT5 : int 0 0 1000 1069 689 1000 13750 1687 1000 1122 ...
## $ PAY_AMT6 : int 0 2000 5000 1000 679 800 13770 1542 1000 0 ...
## $ default  : chr  "Y" "Y" "N" "N" ...
```

```
names(df_a) <- trimws(names(df_a))
```

```
cat("\nValue Counts for 'default' column:\n")
```

```
##
## Value Counts for 'default' column:
```

```
print(table(df_a$default))
```

```
##
##      N      Y
## 23364  6636
```

```
cat("\nMissing Values per Column:\n")
```

```
##
## Missing Values per Column:
```

```
print(colSums(is.na(df_a)))
```

```
##      ID LIMIT_BAL      SEX EDUCATION  MARRIAGE      AGE      PAY_0      PAY_2
##      0          0          0          0          0          0          0          0
##      PAY_3      PAY_4      PAY_5      PAY_6 BILL_AMT1 BILL_AMT2 BILL_AMT3 BILL_AMT4
##      0          0          0          0          0          0          0          0
## BILL_AMT5 BILL_AMT6 PAY_AMT1 PAY_AMT2 PAY_AMT3 PAY_AMT4 PAY_AMT5 PAY_AMT6
##      0          0          0          0          0          0          0          0
## default
##      0
```

```
df_a <- df_a %>% dplyr::select(-ID)
```

```

# Categorical Data Encoding -----
cat_data_cols <- names(df_a)[sapply(df_a, is.character)]
cat_data_cols

## [1] "SEX"          "EDUCATION" "MARRIAGE"  "default"

for (col in cat_data_cols) {
  df_a[[col]] <- as.numeric(factor(df_a[[col]])) - 1
}

cat("\nData Head after Encoding:\n")

##
## Data Head after Encoding:
print(head(df_a))

##   LIMIT_BAL SEX EDUCATION MARRIAGE AGE PAY_0 PAY_2 PAY_3 PAY_4 PAY_5 PAY_6
## 1    20000  0         4         1  24     2     2    -1    -1    -2    -2
## 2   120000  0         4         3  26    -1     2     0     0     0     2
## 3    90000  0         4         3  34     0     0     0     0     0     0
## 4    50000  0         4         1  37     0     0     0     0     0     0
## 5    50000  1         4         1  57    -1     0    -1     0     0     0
## 6    50000  1         1         3  37     0     0     0     0     0     0
##   BILL_AMT1 BILL_AMT2 BILL_AMT3 BILL_AMT4 BILL_AMT5 BILL_AMT6 PAY_AMT1 PAY_AMT2
## 1     3913     3102      689         0         0         0         0      689
## 2     2682     1725     2682     3272     3455     3261         0     1000
## 3    29239    14027    13559    14331    14948    15549    1518     1500
## 4    46990    48233    49291    28314    28959    29547    2000     2019
## 5     8617     5670    35835    20940    19146    19131    2000    36681
## 6    64400    57069    57608    19394    19619    20024    2500     1815
##   PAY_AMT3 PAY_AMT4 PAY_AMT5 PAY_AMT6 default
## 1         0         0         0         0         1
## 2        1000        1000         0        2000         1
## 3        1000        1000        1000        5000         0
## 4        1200        1100        1069        1000         0
## 5       10000        9000         689         679         0
## 6         657        1000        1000         800         0

X <- df_a %>% dplyr::select(-default)
y <- as.factor(df_a$default)

# Data Splitting
set.seed(42)
train_index <- createDataPartition(y, p = 0.8, list = FALSE)
X_train <- X[train_index, ]
X_test <- X[-train_index, ]
y_train <- y[train_index]
y_test <- y[-train_index]

# Scaling Numerical Features (MinMaxScaler equivalent)
preproc_scaler <- preProcess(X_train, method = c("range"))
X_train_scaled <- predict(preproc_scaler, X_train)
X_test_scaled <- predict(preproc_scaler, X_test)

# Convert specific encoded categorical features back to factors for modeling

```

```

# This aligns with the instructor's practice for handling categorical predictors in glm
categorical_cols_to_factor <- c("SEX", "EDUCATION", "MARRIAGE")

for (col in categorical_cols_to_factor) {
  if (col %in% names(X_train_scaled)) {
    X_train_scaled[[col]] <- as.factor(X_train_scaled[[col]])
  }
  if (col %in% names(X_test_scaled)) {
    X_test_scaled[[col]] <- as.factor(X_test_scaled[[col]])
  }
}

# Feature Selection using Correlation -----
# Calculate correlation between each feature in X_train_scaled and y_train
# Ensure y_train is numeric for correlation calculation
y_train_numeric_for_corr <- as.numeric(as.character(y_train))

# For correlation, we should use numeric columns only.
# Temporarily convert factor columns to numeric for correlation calculation.
X_train_scaled_numeric_for_corr <- X_train_scaled
for (col in categorical_cols_to_factor) {
  if (col %in% names(X_train_scaled_numeric_for_corr)) {
    X_train_scaled_numeric_for_corr[[col]] <- as.numeric(as.character(X_train_scaled_numeric_for_corr[[col]]))
  }
}

# Initialize a data frame to store feature scores
feature_scores_df <- data.frame(
  feature = names(X_train_scaled_numeric_for_corr),
  feature_score = NA
)

# Calculate absolute correlation for each feature
for (i in seq_along(names(X_train_scaled_numeric_for_corr))) {
  feature_name <- names(X_train_scaled_numeric_for_corr)[i]
  # Use abs() for absolute correlation to consider strength regardless of direction
  feature_scores_df$feature_score[i] <- abs(cor(X_train_scaled_numeric_for_corr[[feature_name]], y_train_numeric_for_corr))
}

feature_scores_df <- feature_scores_df %>%
  arrange(desc(feature_score))

cat("\nFeature Scores from Correlation:\n")

##
## Feature Scores from Correlation:
print(feature_scores_df)

##      feature feature_score
## 1    PAY_0    0.322446634
## 2    PAY_2    0.257392711
## 3    PAY_3    0.230469913
## 4    PAY_4    0.212689540

```

```
## 5      PAY_5      0.199932267
## 6      PAY_6      0.181199346
## 7  LIMIT_BAL      0.150051768
## 8    PAY_AMT1      0.077391652
## 9    PAY_AMT3      0.059999484
## 10   PAY_AMT4      0.057265595
## 11   PAY_AMT2      0.056237303
## 12   PAY_AMT6      0.052680968
## 13   PAY_AMT5      0.051761604
## 14      SEX      0.039167335
## 15  EDUCATION      0.036829555
## 16  MARRIAGE      0.025302527
## 17  BILL_AMT1      0.019789864
## 18  BILL_AMT2      0.013929878
## 19      AGE      0.013749944
## 20  BILL_AMT3      0.012896992
## 21  BILL_AMT4      0.008805831
## 22  BILL_AMT5      0.006268141
## 23  BILL_AMT6      0.004081589
```

```
# Select features with correlation score > 0.01 (this threshold can be adjusted)
selected_features <- feature_scores_df %>%
  filter(feature_score > 0.01) %>%
  pull(feature)

cat("\nSelected Features:\n")
```

```
##
```

```
## Selected Features:
```

```
print(selected_features)
```

```
## [1] "PAY_0"      "PAY_2"      "PAY_3"      "PAY_4"      "PAY_5"      "PAY_6"
## [7] "LIMIT_BAL"  "PAY_AMT1"   "PAY_AMT3"   "PAY_AMT4"   "PAY_AMT2"   "PAY_AMT6"
## [13] "PAY_AMT5"   "SEX"        "EDUCATION"  "MARRIAGE"   "BILL_AMT1"  "BILL_AMT2"
## [19] "AGE"        "BILL_AMT3"
```

```
X_train_selected <- X_train_scaled %>% dplyr::select(all_of(selected_features))
X_test_selected <- X_test_scaled %>% dplyr::select(all_of(selected_features))
```

```
# Utility Function for Plotting ROC Curve
```

```
plot_probabilities <- function(X_test, y_test, model, model_name = "Model") {
  if (inherits(model, "glm")) {
    pred_probs <- predict(model, newdata = as.data.frame(X_test), type = "response")
  } else if (inherits(model, "rpart")) {
    pred_probs <- predict(model, newdata = as.data.frame(X_test), type = "prob")[, "1"]
  } else if (inherits(model, "probit_model_MASS")) {
    pred_probs <- predict(model, newdata = as.data.frame(X_test), type = "response")
  } else {
    stop("Unsupported model type for plot_probabilities.")
  }
}
```

```
pred_roc <- prediction(pred_probs, y_test)
perf_roc <- performance(pred_roc, "tpr", "fpr")
auc_value <- as.numeric(performance(pred_roc, "auc")@y.values)
```



```

plot(perf_roc,
     col = "blue",
     main = paste("Receiver Operating Characteristic (ROC) Curve |", model_name),
     xlab = "False Positive Rate",
     ylab = "True Positive Rate")
abline(a = 0, b = 1, col = "red", lty = 2)
legend("bottomright", legend = paste("ROC curve (area =", round(auc_value, 2), ")"),
      col = "blue", lty = 1, cex = 0.8)
}

# Logistic Regression Model -----
cat("\n--- Logistic Regression Model ---\n")

##
## --- Logistic Regression Model ---
train_data_lr <- cbind(X_train_selected, default = y_train)

model_lr <- glm(default ~ ., data = train_data_lr, family = binomial(link = "logit"))

y_preds_lr_prob <- predict(model_lr, newdata = X_test_selected, type = "response")
y_preds_lr_class <- factor(ifelse(y_preds_lr_prob > 0.5, 1, 0), levels = levels(y_test))

conf_matrix_lr <- confusionMatrix(y_preds_lr_class, y_test)
cat("\nConfusion Matrix for Logistic Regression:\n")

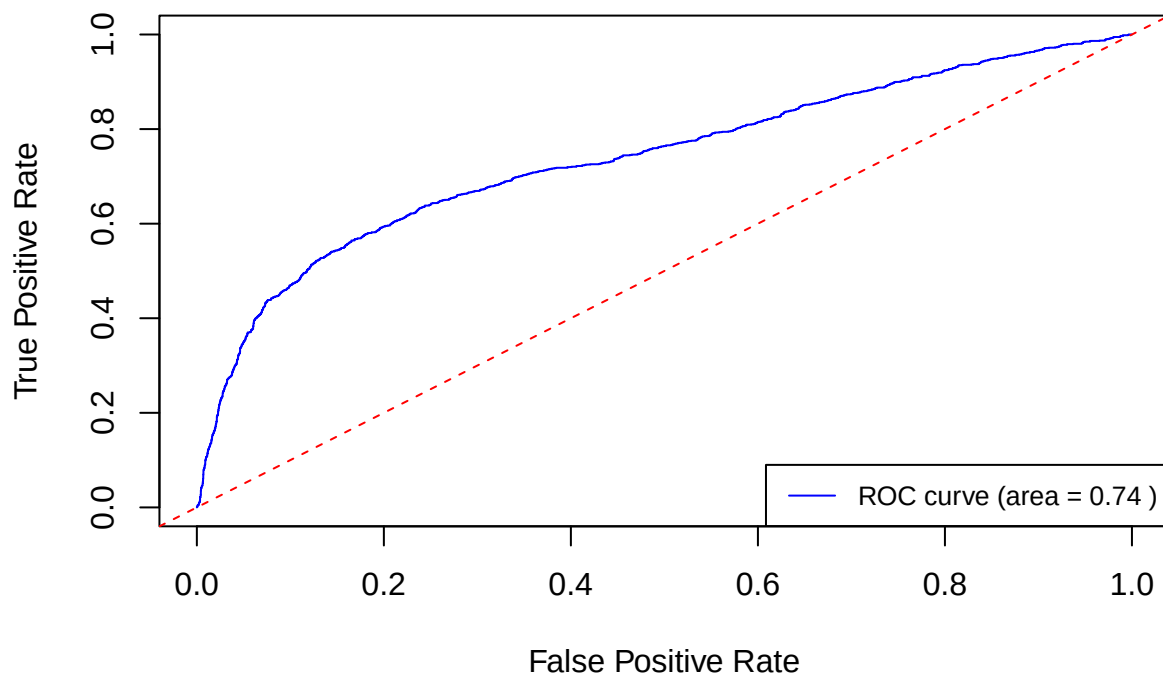
##
## Confusion Matrix for Logistic Regression:
print(conf_matrix_lr)

## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0 4536  998
##           1  136  329
##
##           Accuracy : 0.811
##           95% CI : (0.8008, 0.8208)
##           No Information Rate : 0.7788
##           P-Value [Acc > NIR] : 5.412e-10
##
##           Kappa : 0.2851
##
## Mcnemar's Test P-Value : < 2.2e-16
##
##           Sensitivity : 0.9709
##           Specificity : 0.2479
##           Pos Pred Value : 0.8197
##           Neg Pred Value : 0.7075
##           Prevalence : 0.7788
##           Detection Rate : 0.7561
##           Detection Prevalence : 0.9225
##           Balanced Accuracy : 0.6094

```

```
##
##      'Positive' Class : 0
##
plot_probabilities(X_test_selected, y_test, model_lr, "Logistic Regression")
```

Receiver Operating Characteristic (ROC) Curve | Logistic Regression



```
# Decision Tree Model -----
cat("\n--- Decision Tree Model ---\n")

##
## --- Decision Tree Model ---
train_data_dt <- cbind(X_train_selected, default = y_train)

model_dt <- rpart(default ~ ., data = train_data_dt, method = "class", control = rpart.control(minsplit

y_preds_dt_prob <- predict(model_dt, newdata = X_test_selected, type = "prob")[, "1"]
y_preds_dt_class <- factor(ifelse(y_preds_dt_prob > 0.5, 1, 0), levels = levels(y_test))

conf_matrix_dt <- confusionMatrix(y_preds_dt_class, y_test)
cat("\nConfusion Matrix for Decision Tree:\n")

##
## Confusion Matrix for Decision Tree:
print(conf_matrix_dt)

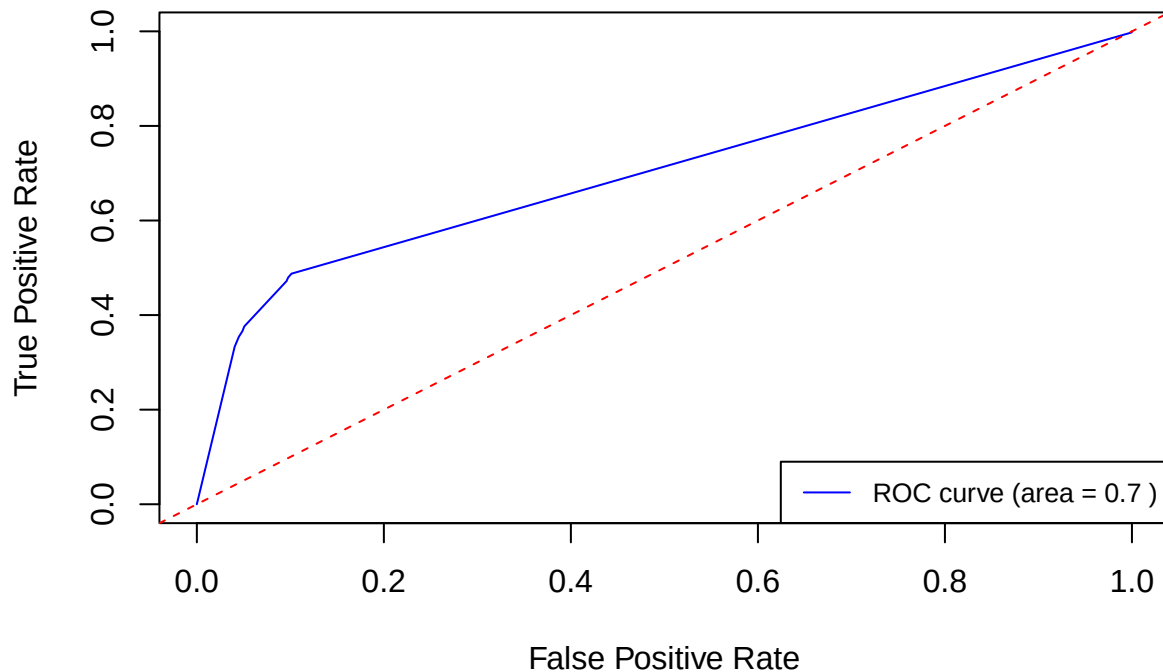
## Confusion Matrix and Statistics
##
```

```

##           Reference
## Prediction    0    1
##           0 4444  841
##           1  228  486
##
##           Accuracy : 0.8218
##           95% CI : (0.8119, 0.8314)
##           No Information Rate : 0.7788
##           P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.3803
##
## Mcnemar's Test P-Value : < 2.2e-16
##
##           Sensitivity : 0.9512
##           Specificity : 0.3662
##           Pos Pred Value : 0.8409
##           Neg Pred Value : 0.6807
##           Prevalence : 0.7788
##           Detection Rate : 0.7408
##           Detection Prevalence : 0.8810
##           Balanced Accuracy : 0.6587
##
##           'Positive' Class : 0
##
plot_probabilities(X_test_selected, y_test, model_dt, "Decision Tree")

```

Receiver Operating Characteristic (ROC) Curve | Decision Tree



```
# Comparison of Models -----  
  
# Function to parse and format classification report metrics  
parse_clf_report <- function(conf_matrix, model_name, y_test_true) {  
  # Extract the confusion matrix table  
  table <- conf_matrix$table  
  
  # Get class labels from the confusion matrix  
  classes <- colnames(table)  
  
  # Calculate metrics for each class  
  metrics_list <- lapply(classes, function(cls) {  
    if (cls == "0") {  
      TP <- table["0", "0"] # True Positives (correctly predicted 0s)  
      FN <- table["0", "1"] # False Negatives (0s predicted as 1s)  
      FP <- table["1", "0"] # False Positives (1s predicted as 0s)  
      TN <- table["1", "1"] # True Negatives (correctly predicted 0s)  
    } else { # cls == "1"  
      TP <- table["1", "1"] # True Positives (correctly predicted 1s)  
      FN <- table["1", "0"] # False Negatives (1s predicted as 0s)  
      FP <- table["0", "1"] # False Positives (0s predicted as 1s)  
      TN <- table["0", "0"] # True Negatives (correctly predicted 0s)  
    }  
  
    precision <- TP / (TP + FP)  
    recall <- TP / (TP + FN)  
  })  
}
```

```

f1_score <- 2 * (precision * recall) / (precision + recall)

# Handle NaN for cases where (TP + FP) or (TP + FN) is zero
if (is.nan(precision)) precision <- 0
if (is.nan(recall)) recall <- 0
if (is.nan(f1_score)) f1_score <- 0

support <- sum(y_test_true == cls)

data.frame(
  model = model_name,
  class = cls,
  precision = precision,
  recall = recall,
  f1_score = f1_score,
  support = support
)
})

# Combine results into a single data frame
do.call(rbind, metrics_list)
}

lr_clf_df <- parse_clf_report(conf_matrix_lr, "Logistic Regression", y_test)
dt_clf_df <- parse_clf_report(conf_matrix_dt, "Decision Tree Classifier", y_test)

comparison_df <- rbind(lr_clf_df, dt_clf_df)

cat("\n--- Model Comparison ---\n")

##
## --- Model Comparison ---

print(comparison_df)

##
##          model class precision    recall  f1_score support
## 1   Logistic Regression      0 0.9708904 0.8196603 0.8888889    4672
## 2   Logistic Regression      1 0.2479277 0.7075269 0.3671875    1327
## 3 Decision Tree Classifier      0 0.9511986 0.8408704 0.8926383    4672
## 4 Decision Tree Classifier      1 0.3662396 0.6806723 0.4762371    1327

# Part B: Probit Model -----
cat("\n--- Probit Regression Model ---\n")

##
## --- Probit Regression Model ---

y_train_numeric <- as.numeric(as.character(y_train))

train_data_pt <- cbind(X_train_selected, default = y_train_numeric)

model_pt <- glm(default ~ ., data = train_data_pt, family = binomial(link = "probit"))

## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred

```

```

class(model_pt) <- c("probit_model_MASS", class(model_pt))

cat("\nProbit Regression Results Summary:\n")

##
## Probit Regression Results Summary:
print(summary(model_pt))

##
## Call:
## glm(formula = default ~ ., family = binomial(link = "probit"),
##      data = train_data_pt)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)    -5.75659    24.26355  -0.237  0.812461
## PAY_0           3.14561     0.11373  27.659 < 2e-16 ***
## PAY_2           0.41480     0.13326   3.113  0.001854 **
## PAY_3           0.37487     0.14601   2.567  0.010248 *
## PAY_4           0.13164     0.16149   0.815  0.414968
## PAY_5           0.21273     0.17532   1.213  0.224995
## PAY_6          -0.02982     0.14228  -0.210  0.834004
## LIMIT_BAL      -0.26523     0.07617  -3.482  0.000497 ***
## PAY_AMT1       -3.47526     0.62888  -5.526  3.27e-08 ***
## PAY_AMT3       -0.55629     0.36332  -1.531  0.125737
## PAY_AMT4       -0.71619     0.45046  -1.590  0.111855
## PAY_AMT2       -6.83323     1.76331  -3.875  0.000107 ***
## PAY_AMT6       -0.52804     0.38611  -1.368  0.171447
## PAY_AMT5       -0.31668     0.34586  -0.916  0.359857
## SEX1           0.05964     0.01954   3.053  0.002268 **
## EDUCATION0.2    3.76521    24.26167   0.155  0.876670
## EDUCATION0.4    3.74525    24.26168   0.154  0.877319
## EDUCATION0.6    3.11920    24.26261   0.129  0.897706
## EDUCATION0.8    3.73945    24.26167   0.154  0.877507
## EDUCATION1      3.14235    24.26197   0.130  0.896948
## MARRIAGE0.333333333333333 0.66572     0.29029   2.293  0.021830 *
## MARRIAGE0.666666666666667 0.64741     0.30253   2.140  0.032353 *
## MARRIAGE1       0.57458     0.29041   1.979  0.047871 *
## BILL_AMT1      -2.46554     0.57719  -4.272  1.94e-05 ***
## BILL_AMT2       0.44436     0.71791   0.619  0.535939
## AGE            0.19409     0.06465   3.002  0.002680 **
## BILL_AMT3       2.55872     1.12453   2.275  0.022885 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 25365  on 24000  degrees of freedom
## Residual deviance: 22492  on 23974  degrees of freedom
## AIC: 22546
##
## Number of Fisher Scoring iterations: 11

```

```

y_preds_probs_pt <- predict(model_pt, newdata = X_test_selected, type = "response")
y_preds_pt_class <- factor(ifelse(y_preds_probs_pt > 0.5, 1, 0), levels = levels(y_test))

conf_matrix_pt <- confusionMatrix(y_preds_pt_class, y_test)
cat("\nConfusion Matrix for Probit Model:\n")

##
## Confusion Matrix for Probit Model:
print(conf_matrix_pt)

## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0 4568 1060
##           1  104  267
##
##           Accuracy : 0.806
##           95% CI : (0.7957, 0.8159)
##           No Information Rate : 0.7788
##           P-Value [Acc > NIR] : 1.445e-07
##
##           Kappa : 0.2411
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
##           Sensitivity : 0.9777
##           Specificity : 0.2012
##           Pos Pred Value : 0.8117
##           Neg Pred Value : 0.7197
##           Prevalence : 0.7788
##           Detection Rate : 0.7615
##           Detection Prevalence : 0.9382
##           Balanced Accuracy : 0.5895
##
##           'Positive' Class : 0
##
plot_probabilities(X_test_selected, y_test, model_pt, "Probit Model")

```

Receiver Operating Characteristic (ROC) Curve | Probit Model

